



SEP2025

DRONE CHOREO "SKYORCHESTRATOR"

Software-Entwicklungspraktikum (SEP)
Sommersemester 2025

Fachentwurf

Auftraggeber
Technische Universität Braunschweig
Institut für Robotik und Prozessinformatik
Prof. Dr. Jochen J. Steil
Mühlenpfordtstraße 23
38106 Braunschweig

Betreuer: Kilian Klankers

Auftragnehmer:

Name	E-Mail-Adresse
Mohammed Alhalabi	m.alhalabi@tu-braunschweig.de
Luan Hyseni	l.hyseni@tu-braunschweig.de
Mohamed Sabri Jlizi	m.jlizi@tu-braunschweig.de
Mohammad Khatib	m.khatib@tu-braunschweig.de
Lucas Neidahl	l.neidahl@tu-braunschweig.de
Marvin Voigt	marvin.voigt@tu-braunschweig.de

Braunschweig, 4. Juni 2025

Bearbeiterübersicht

Kapitel	Autoren	Kommentare
1	Luan Hyseni	fertig
1.1	Luan Hyseni, Mohamed Alhalabi	Fertig
2	Lucas Neidahl	Fertig
2.1	Lucas Neidahl	Fertig
2.2	Lucas Neidahl	Fertig
2.3	Lucas Neidahl	Fertig
2.4	Lucas Neidahl	Fertig
2.5	Lucas Neidahl	Fertig
2.6	Lucas Neidahl	Fertig
2.7	Mohammed Alhalabi	Fertig
2.8	Mohammed Alhalabi	Fertig
3	Mohamed Sabri Jlizi, Mohammad Khatib	fertig
3.1	Mohamed Sabri Jlizi, Mohammad Khatib	fertig
3.2	Mohamed Sabri Jlizi, Mohammad Khatib	fertig
4	Marvin Voigt	Fertig
4.1	Marvin Voigt	Fertig
4.2	Marvin Voigt	Fertig
4.3	Marvin Voigt	Fertig
4.4	Marvin Voigt, Mohamed Alhalabi	Fertig
4.5	Marvin Voigt	Fertig
4.6	Marvin Voigt, Mohamed Alhalabi	Fertig
5	Mohammed Alhalabi	Fertig

Inhaltsverzeichnis

1	Einleitung	5
1.1	Projektdetails	6
1.1.1	Projektzyklus in SkyOrchestrator	7
1.1.2	Szenenkonfiguration	9
2	Analyse der Produktfunktionen	11
2.1	Analyse von Funktion F10: Konfiguration der Show	11
2.2	Analyse von Funktion F11: Szene anlegen und Metadaten erfassen	12
2.3	Analyse von Funktion F12: Formation und Bewegung definieren	13
2.4	Analyse von Funktion F13: Licht- und Zeiteffekte konfigurieren	14
2.5	Analyse von Funktion F14: Szene speichern und zur Simulation übergeben	15
2.6	Analyse von Funktion F20: Choreographie simulieren	16
2.7	Analyse von Funktion F30: Projekt verwalten	16
2.8	Analyse von Funktion F40: Szenenprüfung auf Eingabefehler	18
3	Datenmodell	19
3.1	Diagramm	19
3.2	Erläuterung der Zusammenhänge der Klassen	20
4	Konfiguration	23
4.1	Technische Voraussetzungen	23
4.2	Projektstruktur	24
4.3	Start und Nutzung der Anwendung	24
4.4	Konfigurationsdateien	25
4.5	Konfiguration über die Benutzeroberfläche	26
4.6	Mögliche Erweiterungen	26
5	Glossar	27

Abbildungsverzeichnis

1.1	Zustandsdiagramm des Projektlebenszyklus	7
1.2	Aktivitätsdiagramm : Projektzyklus in <i>SkyOrchestrator</i>	8
1.3	Zustandsdiagramm der Szenenkonfiguration	9
2.1	Sequenzdiagramm für F10	11
2.2	Sequenzdiagramm für F11	12
2.3	Sequenzdiagramm für F12	13
2.4	Sequenzdiagramm für F13	14
2.5	Sequenzdiagramm für F14	15
2.6	Sequenzdiagramm für F20	16
2.7	Sequenzdiagramm für F14	17
2.8	Ein beispielhaftes Sequenzdiagramm	18
3.1	Ein beispielhaftes Klassendiagramm	19

1 Einleitung

Dieses Dokument beschreibt den fachlichen Entwurf der Softwarelösung SkyOrchestrator, die im Rahmen des Softwareentwicklungspraktikums im Sommersemester 2025 an der Technischen Universität Braunschweig entwickelt wird. Es bietet einen strukturierten Überblick über die Motivation, die Zielsetzung, die Systemkomponenten und die Benutzerabläufe der geplanten Anwendung.

SkyOrchestrator stellt eine moderne, kreative und nachhaltige Alternative zu herkömmlichen Feuerwerken dar. Mithilfe von Lichtdrohnen, die durch koordinierte Flugbewegungen und farbige Lichteffekte visuelle Muster erzeugen, lassen sich komplexe Shows definieren, abspeichern und in einer 3D-Simulationsumgebung darstellen. Entwickelt wird eine intuitiv bedienbare Anwendung, die auch von Personen ohne technische Vorkenntnisse genutzt werden kann. Im Vordergrund steht die einfache Erstellung und Bearbeitung von sogenannten Szenen, in denen Parameter wie Drohnenanzahl, Flugformationen, Lichteffekte und zeitliche Abläufe konfiguriert werden. Die Zielgruppen des Systems umfassen unter anderem Eventveranstaltende, Kreativagenturen, Bildungseinrichtungen sowie interessierte Privatpersonen. Neben dem gestalterischen Anspruch verfolgt das Projekt das Ziel, eine umweltfreundliche Alternative zu traditionellen pyrotechnischen Shows aufzuzeigen. Durch die digitale Planung und visuelle Simulation werden Risiken vermieden und Ressourcen geschont. Nutzerinnen und Nutzer sollen ihre Ideen visuell umsetzen, validieren und schrittweise verfeinern können – bevor eine reale Drohnenshow tatsächlich stattfindet. Die Anwendung ist modular aufgebaut. Sie umfasst unter anderem ein grafisches Benutzerinterface zur Projektverwaltung und Szenenerstellung, ein Konfigurationsmodul zur Parametrisierung der Drohneneigenschaften, ein Simulationsmodul auf Basis von PyBullet zur Echtzeitdarstellung. Technisch basiert SkyOrchestrator auf Python und nutzt die Simulationsbibliothek PyBullet zur Darstellung der Flugbewegungen. Eine spätere Erweiterung um Exportfunktionen im JSON-Format ist vorgesehen, um externe Verarbeitung oder perspektivisch auch reale Drohnensteuerung vorzubereiten. Im Rahmen des SEP liegt der Fokus jedoch ausschließlich auf der virtuellen Ausführung. Im weiteren Verlauf dieses Dokuments wird zunächst ein Aktivitätsdiagramm zur Verdeutlichung des Benutzerablaufs vorgestellt. Anschließend folgen zwei Zustandsdiagramme, welche den Lebenszyklus eines Projekts sowie die internen Zustandsübergänge innerhalb einer Szene beschreiben. Danach werden die zentralen Produktfunktionen detailliert analysiert und durch Sequenzdiagramme veranschaulicht. Schließlich wird ein Überblick über das Datenmodell sowie die Konfiguration der Anwendung gegeben.

1.1 Projektdetails

In diesem Abschnitt werden zentrale funktionale und technische Aspekte des SkyOrchestrator-Systems erläutert. Die beschriebenen Abläufe und Komponenten sind für das Verständnis und die Umsetzung der Software essenziell. Nach einer überblicksartigen Darstellung des typischen Systemverhaltens folgen detaillierte Erläuterungen zur Szenenkonfiguration und Projektverwaltung. Die dabei dargestellten Zustandsdiagramme dienen der Visualisierung des Ablaufverhaltens innerhalb der Anwendung.

1.1.1 Projektzyklus in SkyOrchestrator

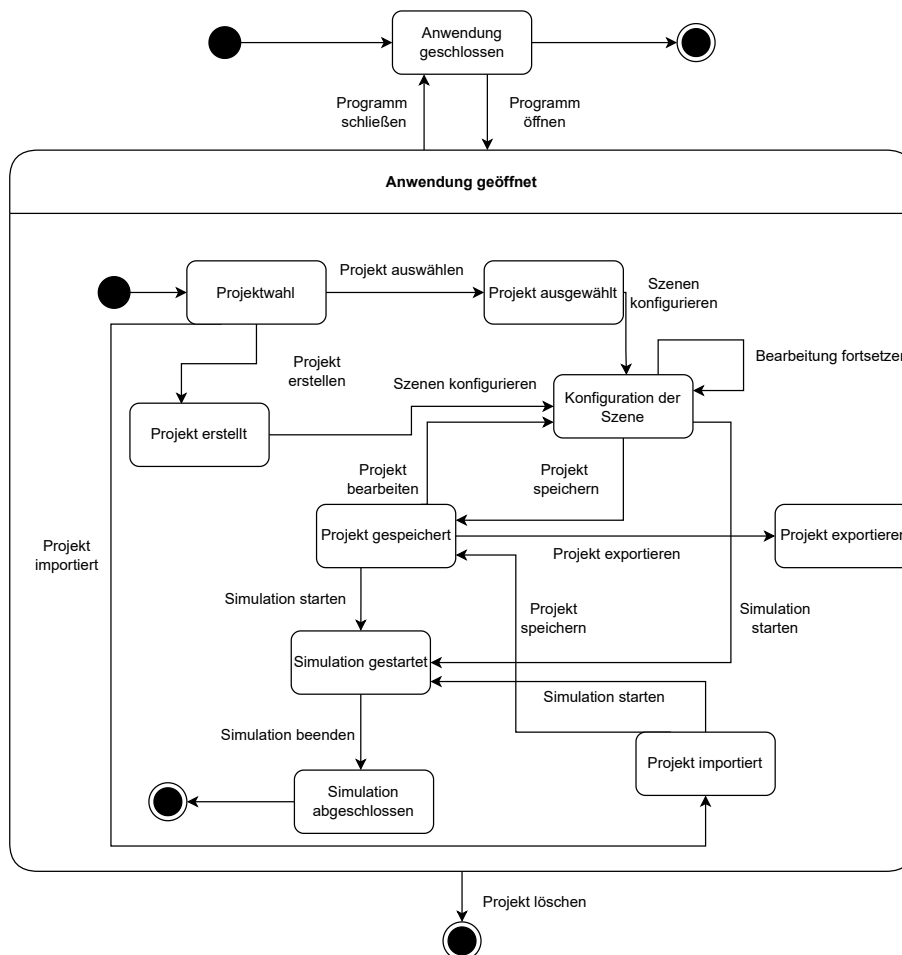
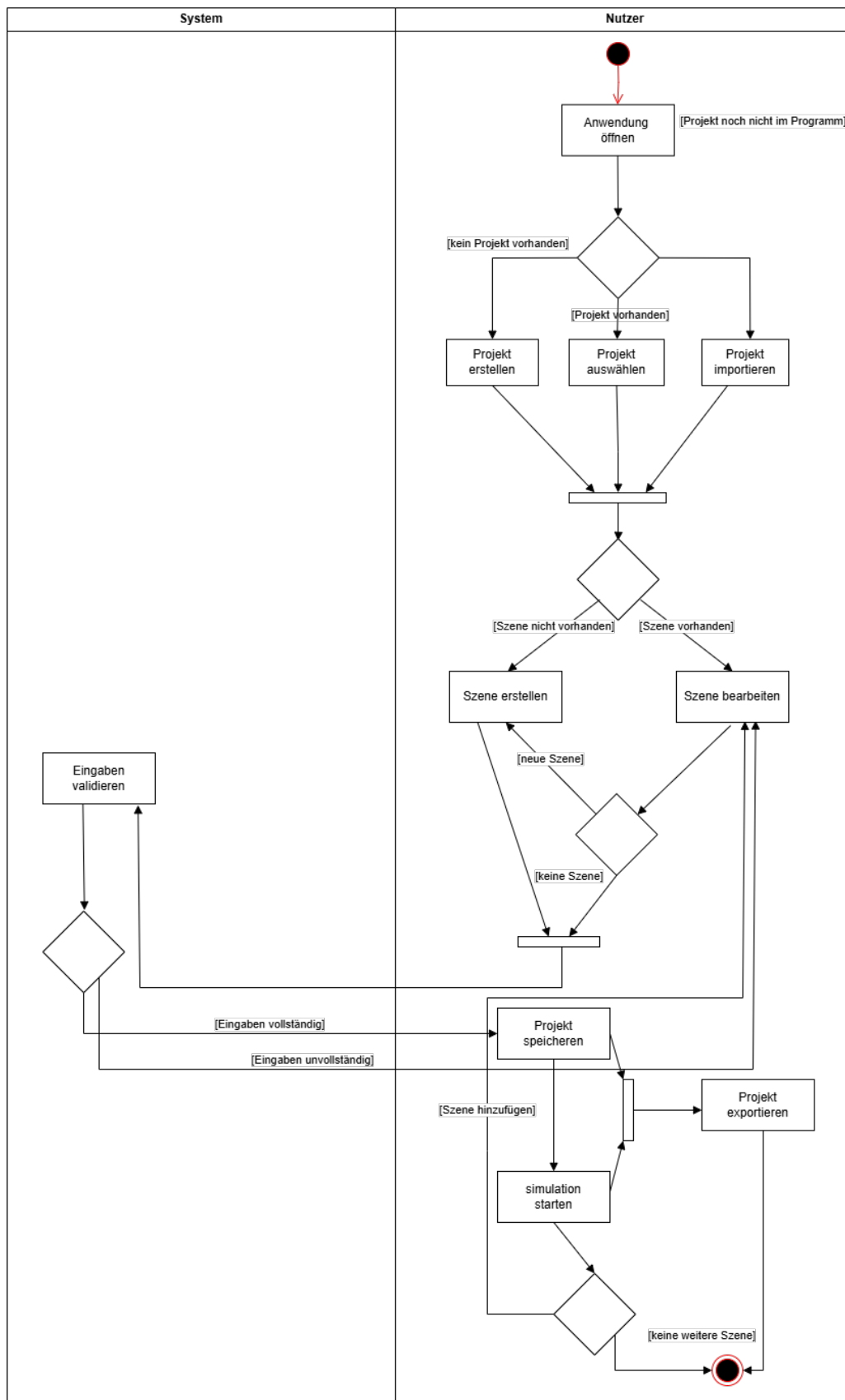


Abbildung 1.1: Zustandsdiagramm des Projektlebenszyklus

Abbildung 1.2: Aktivitätsdiagramm : Projektzyklus in *SkyOrchestrator*

Das erste Zustandsdiagramm beschreibt den typischen Lebenszyklus eines Drohnen-Show-Projekts in der Anwendung SkyOrchestrator. Es beginnt mit dem Öffnen der Anwendung, woraufhin Nutzer entweder ein bestehendes Projekt auswählen oder ein neues Projekt erstellen können. Eine alternative Einstiegsmöglichkeit stellt der Projektimport dar. Sobald ein Projekt ausgewählt oder erstellt wurde, kann die Konfiguration von Szenen erfolgen. Dieser Schritt ist zentral für die Definition der eigentlichen Showinhalte, zum Beispiel Flugformationen oder Lichtparameter. Die Szenenkonfiguration selbst wird im nächsten Abschnitt im Detail dargestellt. Nach der Konfiguration kann das Projekt gespeichert und optional exportiert werden. Die Simulation kann sowohl vor als auch nach dem Speichern durchgeführt werden, was die Flexibilität bei der Entwicklung von Shows erhöht. Projekte können zu jedem Zeitpunkt gelöscht werden. Damit ist auch ein direkter Abbruch des Bearbeitungsprozesses möglich. Der Ablauf eines Projekts im SkyOrchestrator folgt keinem starren linearen Muster. Nutzer:innen können ein Projekt jederzeit erneut öffnen, Szenen ergänzen oder ändern, Simulationen durchführen und Zwischenergebnisse speichern. Dadurch wird ein flexibler Umgang mit dem Projekt ermöglicht, der sowohl iteratives Arbeiten als auch nachträgliche Anpassungen unterstützt.

1.1.2 Szenenkonfiguration

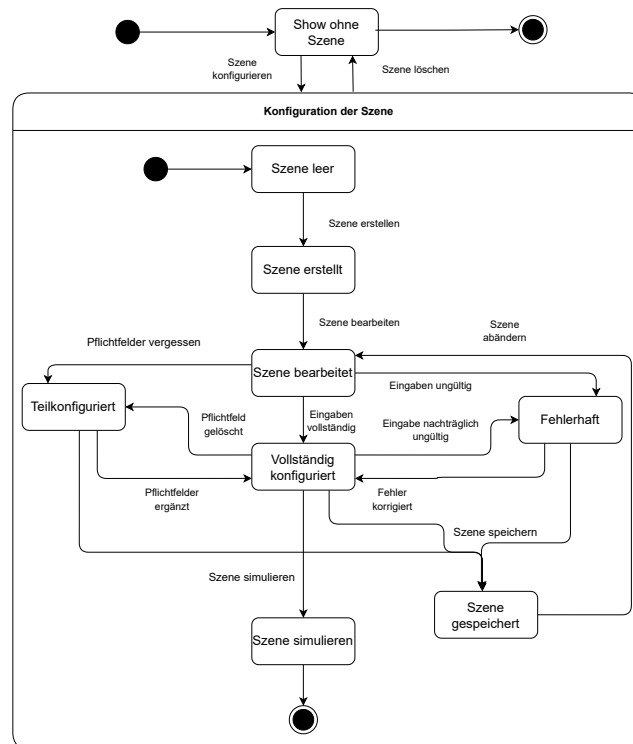


Abbildung 1.3: Zustandsdiagramm der Szenenkonfiguration

Das zweite Zustandsdiagramm stellt den internen Ablauf innerhalb der Szenenkonfiguration dar. Es beschreibt detailliert die möglichen Zustände einer Szene von der Erstellung bis zur Simulation. Der Prozess beginnt mit dem Zustand Szene leer, aus dem heraus eine neue Szene erstellt wird. Nach der Erstellung befindet sich die Szene im Zustand Szene erstellt und kann dann bearbeitet werden. Im Bearbeitungsprozess unterscheidet das System zwischen gültigen und ungültigen Eingaben. Ungültige Eingaben führen zum Zustand Fehlerhaft, während vollständige Eingaben den Zustand Vollständig konfiguriert erreichen. Eine Szene kann auch nur teilkonfiguriert sein, wenn beispielsweise Pflichtfelder fehlen oder gelöscht wurden. In diesem Fall kann sie nach entsprechender Korrektur erneut bearbeitet oder vervollständigt werden. Szenen können unabhängig vom Konfigurationsstand gespeichert und später weiterbearbeitet werden. Eine Simulation ist nur möglich, wenn alle relevanten Eingaben vollständig und korrekt vorliegen. Auch die Rückführung aus fehlerhaften Zuständen zur Bearbeitung ist explizit vorgesehen und modelliert..

Beide Diagramme verdeutlichen, wie der SkyOrchestrator komplexe Abläufe strukturiert und intern abbildet. Sie dienen als Grundlage für die Umsetzung der Benutzerinteraktion sowie der automatischen Validierung innerhalb des Systems.

2 Analyse der Produktfunktionen

Dieses Kapitel analysiert die einzelnen Produktfunktionen aus dem Pflichtenheft, um die spätere Architekturentwicklung zu ermöglichen. Jede Funktion wird in einem eigenen Unterkapitel dokumentiert.

2.1 Analyse von Funktion F10: Konfiguration der Show

Diese Funktion ermöglicht dem Benutzer die Planung der Inhalte einer Drohnenshow durch Erstellung, Bearbeitung und Verwaltung einzelner Szenen. Die Anforderungen RM1, RM2, RM4, RM6 und RM7 sind hierbei zentral. Beim Anlegen einer neuen Szene klickt der Nutzer in der GUI auf "Neue Szene erstellen". Daraufhin wird im Hintergrund der ShowKonfigurator aufgerufen, etwa über eine Methode wie `neueSzeneAnlegen()`. Diese Funktion ruft die ShowKonfigurator-Komponente auf, welche mit Hilfe des Projektmanagement-Moduls ein neues leeres Szenen-Objekt erstellt. Anschließend gibt das Projektmanagement Modul eine Bestätigung. Der Show-Konfigurator informiert daraufhin die GUI, welche die Anzeige aktualisiert und dem Nutzer die neu erstellte Szene und weitere Eingabefelder zeigt, um die Szene nun zu konfigurieren.

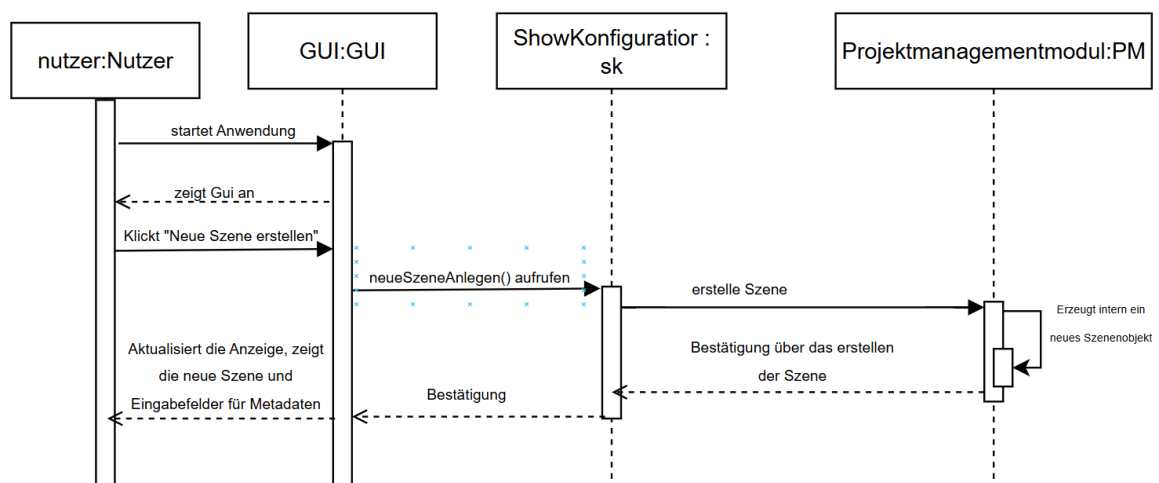


Abbildung 2.1: Sequenzdiagramm für F10

2.2 Analyse von Funktion F11: Szene anlegen und Metadaten erfassen

Beim Anlegen einer neuen Szene wird der Nutzer in der GUI aufgefordert, grundlegende Metadaten wie Name, Dauer und Drohnenanzahl einzugeben. Diese Metadaten werden anschließend vom GUI-Element an den ShowKonfigurator übergeben, der sie nutzt, um eine neue Szene zu initialisieren. Der ShowKonfigurator reicht die Metadaten weiter an das Projektmanagement-Modul, welches ein neues Szenenobjekt erstellt und die übergebenen Informationen speichert. Nach erfolgreicher Anlage gibt das Projektmanagement Modul eine Bestätigung oder das erstellte Objekt zurück, woraufhin die GUI entsprechend aktualisiert wird und dem Nutzer die leere Szene oder weitere Bearbeitungsmöglichkeiten anzeigt.

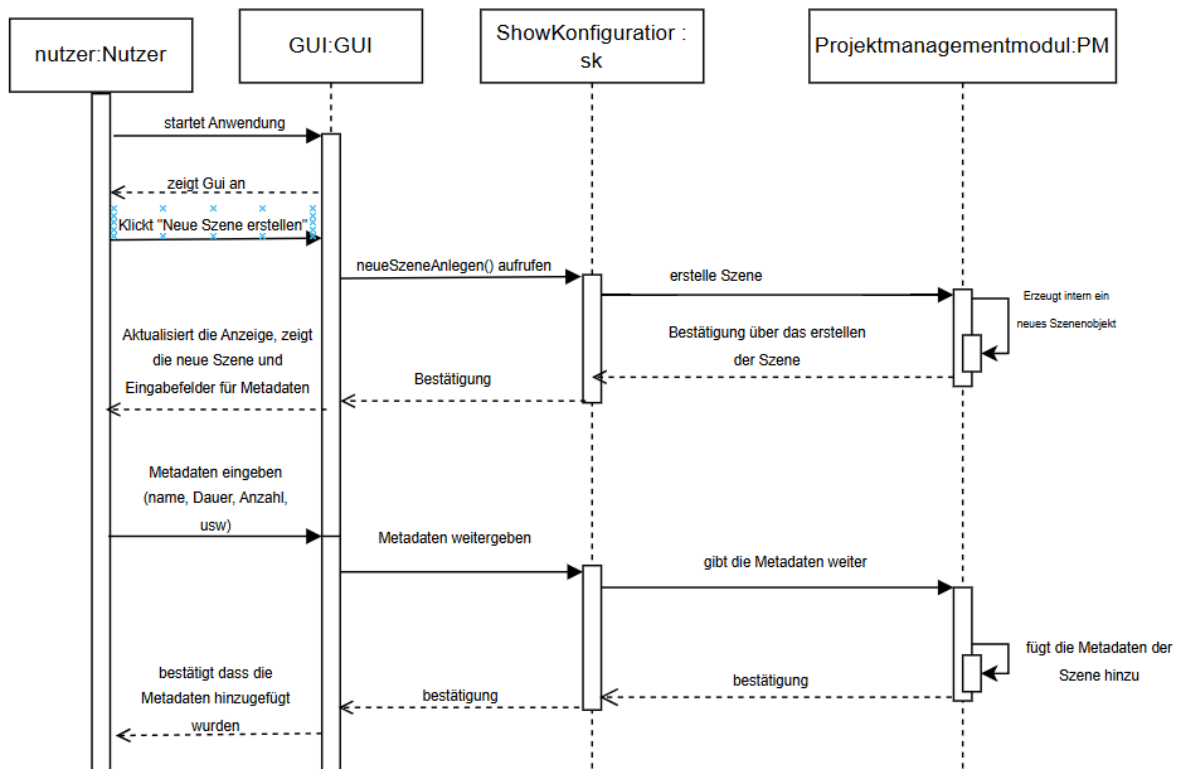


Abbildung 2.2: Sequenzdiagramm für F11

2.3 Analyse von Funktion F12: Formation und Bewegung definieren

Beim Auswählen einer Formation ist eine Szene bereits in der GUI geöffnet und aktiv, die SzenenVerwaltung kennt die aktuelle Drohnenanzahl. Der Nutzer klickt auf „Formation auswählen“, woraufhin die GUI dem ShowKonfigurator die aktuelle Szene übermittelt. Der ShowKonfigurator fragt in der FormationsBibliothek nach verfügbaren Formationen und erhält eine Liste. Diese Optionen werden der GUI angezeigt. Nachdem der Nutzer eine Formation ausgewählt hat, teilt die GUI dem ShowKonfigurator die gewählte Formation mit. Der ShowKonfigurator holt über die SzenenVerwaltung die Szenendetails. Der ShowKonfigurator wendet die Formation an, indem er das Projektmanagement-Modul die Anwendung der Formation übergibt. Das Projektmanagement-Modul berechnet basierend auf Formation und Drohnenanzahl die exakten Positionen jeder Drohne und aktualisiert deren Zustände. Nach Bestätigung der Anwendung aktualisiert der ShowKonfigurator die GUI.

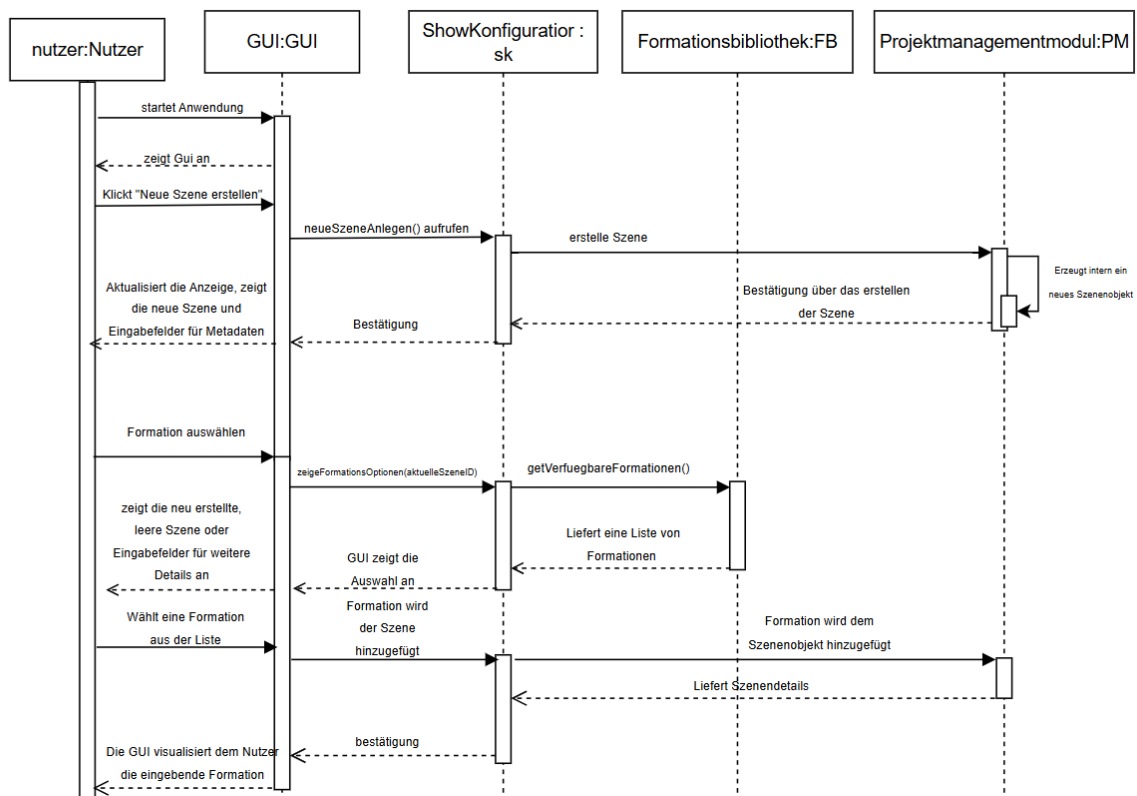


Abbildung 2.3: Sequenzdiagramm für F12

2.4 Analyse von Funktion F13: Licht- und Zeiteffekte konfigurieren

Beim Konfigurieren eines Lichteffekts ist zunächst eine Szene in der GUI geöffnet und enthält bereits Drohnen. Der Nutzer klickt auf „Lichteffekt hinzufügen“ oder wählt in der Timeline bzw. die gewünschten Drohnen aus. Die GUI übermittelt daraufhin an den ShowKonfigurator die aktuelle Szene, die ausgewählten Drohnen und den Zeitpunkt in der Timeline. Der ShowKonfigurator holt über die LichtEffektBibliothek eine Liste verfügbarer Effekte und zeigt diese der GUI an. Nachdem der Nutzer einen Effekt (beispielsweise „Rot Dauerlicht“) samt optionaler Parameter wie Dauer und Helligkeit ausgewählt hat, übergibt die GUI die Daten zurück an den ShowKonfigurator. Anschließend aktualisiert der ShowKonfigurator die GUI, sodass die Timeline und gegebenenfalls eine Vorschau den neu angelegten Lichteffekt anzeigen.

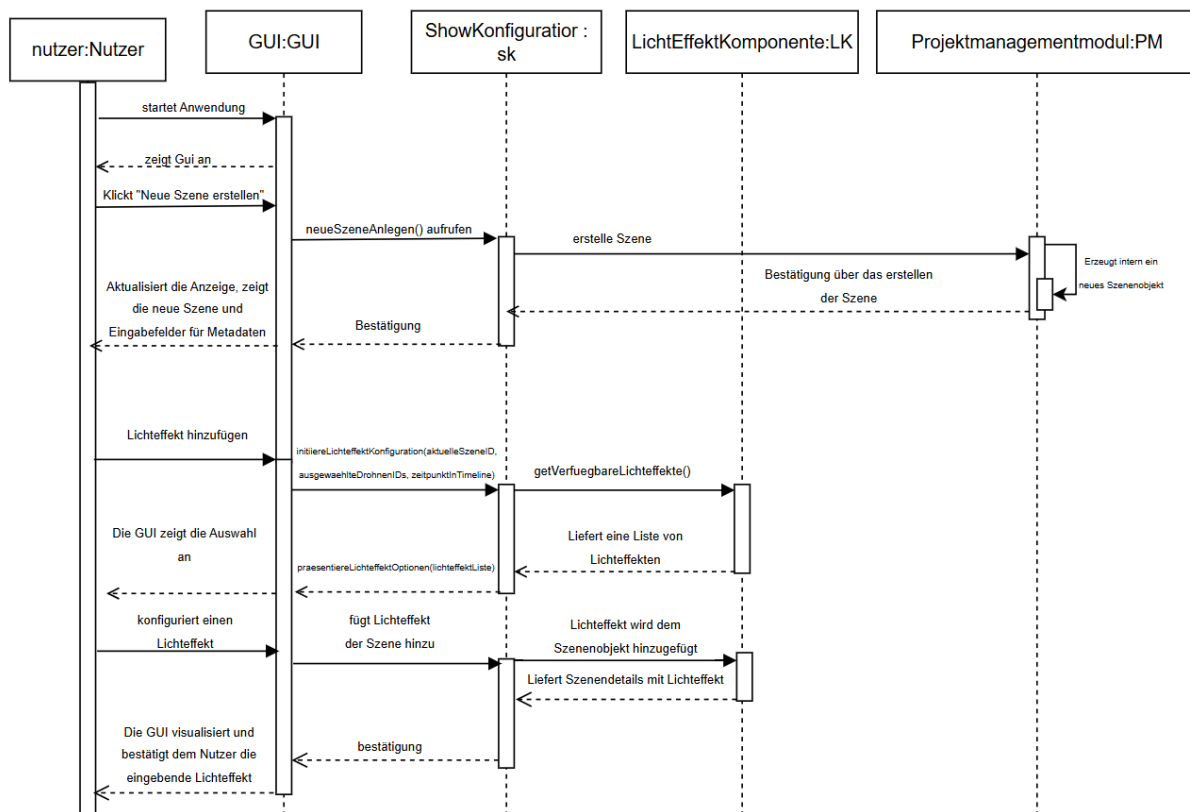


Abbildung 2.4: Sequenzdiagramm für F13

2.5 Analyse von Funktion F14: Szene speichern und zur Simulation übergeben

Die Simulation einer Szene beginnt, wenn der Nutzer in der geöffneten und vollständig eingestellten Szene auf „Simulation starten“ klickt. Die GUI leitet diesen Befehl an den ShowKonfigurator weiter, der daraufhin alle benötigten Szenendaten von dem Projektmanagement-Modul anfordert. Nachdem der ShowKonfigurator die vollständigen Informationen zurückerhalten hat, kann er die Szene, falls gewünscht, zunächst über die SpeicherKomponente sichern. Anschließend bereitet der ShowKonfigurator die Daten in einem simulationsgerechten Format auf und übergibt sie an die Simulationsbibliothek (Pybullet). Sobald die Simulationsbibliothek den Eingang der Szene bestätigt und die Simulation initialisiert hat, informiert der ShowKonfigurator die GUI darüber, dass die Simulation gestartet wurde. Die GUI wechselt daraufhin in den Simulationsmodus und zeigt dem Nutzer an, wie die Drohneninformationen, Bewegungsabläufe und Lichteffekte ablaufen.

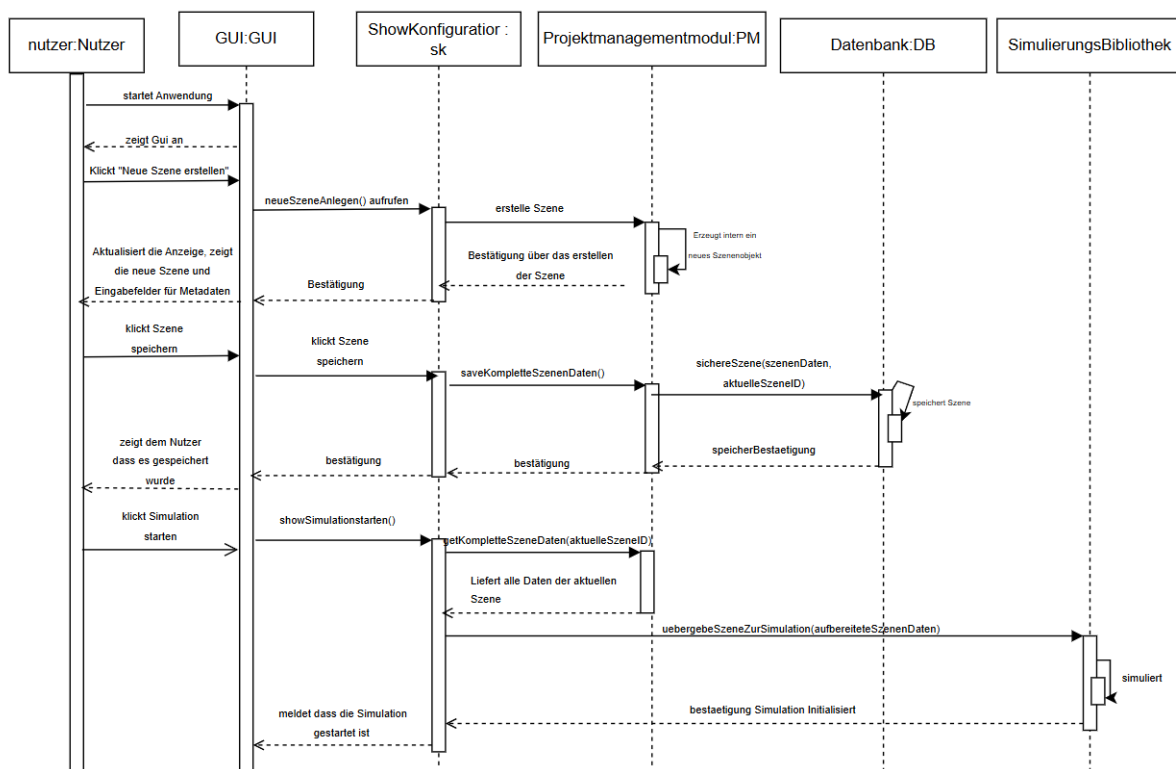


Abbildung 2.5: Sequenzdiagramm für F14

2.6 Analyse von Funktion F20: Choreographie simulieren

Beim Starten der Simulation lädt der Nutzer über die GUI-Modul die gewünschte Show (entweder die aktuell geladene oder eine gespeicherte) zur Anzeige und klickt auf „Show simulieren“. Die GUI reicht den Befehl an das Konfigurationsmodul weiter, das alle relevanten Show-Daten vom Projektmanagement-Modul abrufen. Anschließend übergibt das Konfigurationsmodul diese vollständigen Daten an die Simulationsbibliothek. Dort wird die 3D-Umgebung initialisiert und die Drohnenobjekte entsprechend der ersten Szene positioniert. Sobald das Simulationsmodul die Daten erfolgreich übernommen hat, läuft die Simulation ab. Nachdem es die Simulation beendet hat, meldet es dem Konfigurationsmodul, dass die Show zu Ende ist. Dieses Modul leitet wiederum die Bestätigung an das Gui weiter. Das Gui wird wieder aktiv und der Nutzer kann erneut über das Programm die Show konfigurieren oder erneut simulieren.

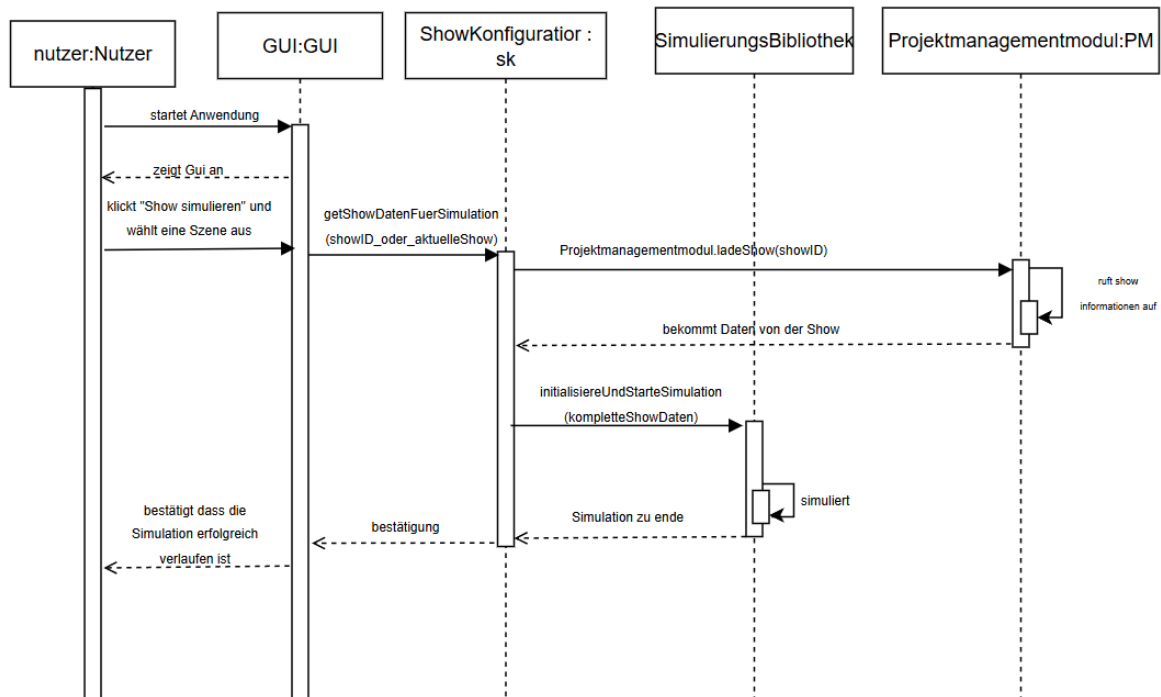


Abbildung 2.6: Sequenzdiagramm für F20

2.7 Analyse von Funktion F30: Projekt verwalten

Die Funktion F30: Projekt verwalten beschreibt das Speichern, Laden und Exportieren von Projekten innerhalb des SkyOrchestrator-Systems. Sobald der Benutzer in der grafischen Oberfläche eine entsprechende Aktion auslöst, beispielsweise über „Projekt speichern“ oder „Projekt laden“,

leitet die GUI den Befehl an den ProjektManager weiter. Dieser übernimmt die Verarbeitung des Befehls, etwa durch Serialisierung der Projektdaten und Weitergabe an das Dateisystem im Falle eines Speichervorgangs oder durch Einlesen und Validierung bestehender Daten beim Laden eines Projekts. Der ProjektManager kommuniziert dabei direkt mit dem Dateisystem, ruft die jeweiligen Methoden auf und gibt nach erfolgreicher oder fehlerhafter Ausführung eine Rückmeldung an die GUI zurück. Diese aktualisiert daraufhin die Benutzeroberfläche und zeigt dem Benutzer das Ergebnis an. Auch Exportvorgänge werden über dieses Modul abgewickelt, indem Projektinformationen in ein geeignetes Austauschformat überführt und gespeichert werden. Die folgende Abbildung zeigt die beteiligten Komponenten sowie die Austauschbeziehungen im Rahmen eines typischen Speichervorgangs.

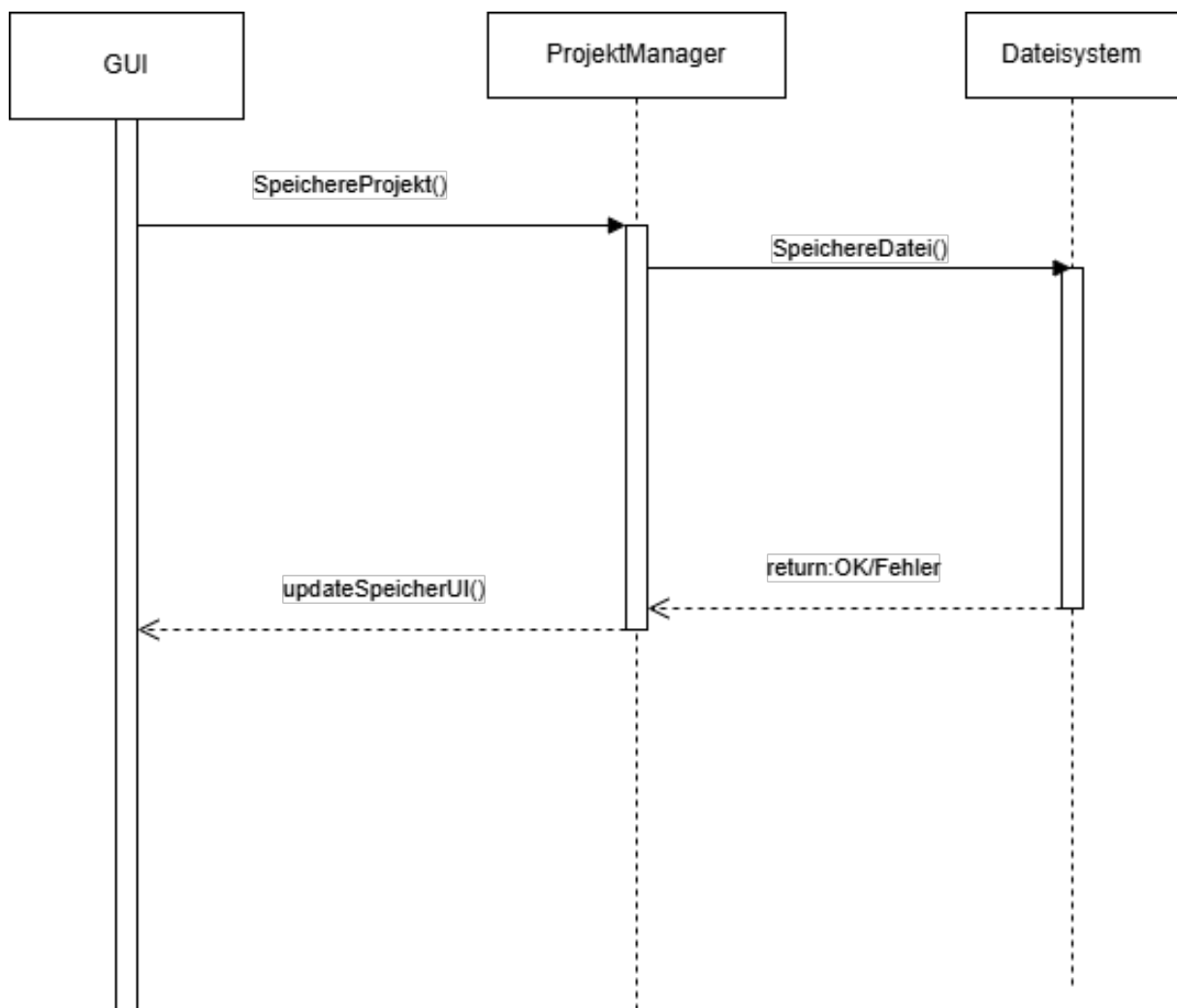


Abbildung 2.7: Sequenzdiagramm für F14

2.8 Analyse von Funktion F40: Szenenprüfung auf Eingabefehler

Die Funktion F40: Szenenprüfung auf Eingabefehler ermöglicht die automatisierte Überprüfung einer konfigurierten Szene auf Vollständigkeit und Plausibilität der eingegebenen Daten. Sobald der Benutzer über die grafische Benutzeroberfläche eine entsprechende Aktion auslöst, etwa durch Auswahl des Befehls „Szene prüfen“ oder durch den Versuch, eine Szene zu simulieren oder zu speichern, übermittelt die GUI eine Prüfanforderung an die Validierungskomponente. Diese greift auf die aktuellen Szenendaten der SzenenVerwaltung zu, analysiert sämtliche Parameter und Eingaben der Szene, und überprüft dabei insbesondere Pflichtfelder, Wertebereiche, zeitliche Abfolgen sowie geometrische Kollisionen oder doppelte Zuordnungen. Im Falle fehlerhafter oder unvollständiger Daten generiert die Validierung eine strukturierte Fehlerliste und sendet diese an die GUI zurück. Die grafische Oberfläche visualisiert die Rückmeldung in geeigneter Form, beispielsweise durch Fehlermarkierungen oder Meldungsfenster, und ermöglicht dem Benutzer die gezielte Korrektur der betroffenen Elemente. Die folgende Abbildung zeigt den Nachrichtenaustausch zwischen den beteiligten Komponenten beim Ablauf einer Szenenprüfung.

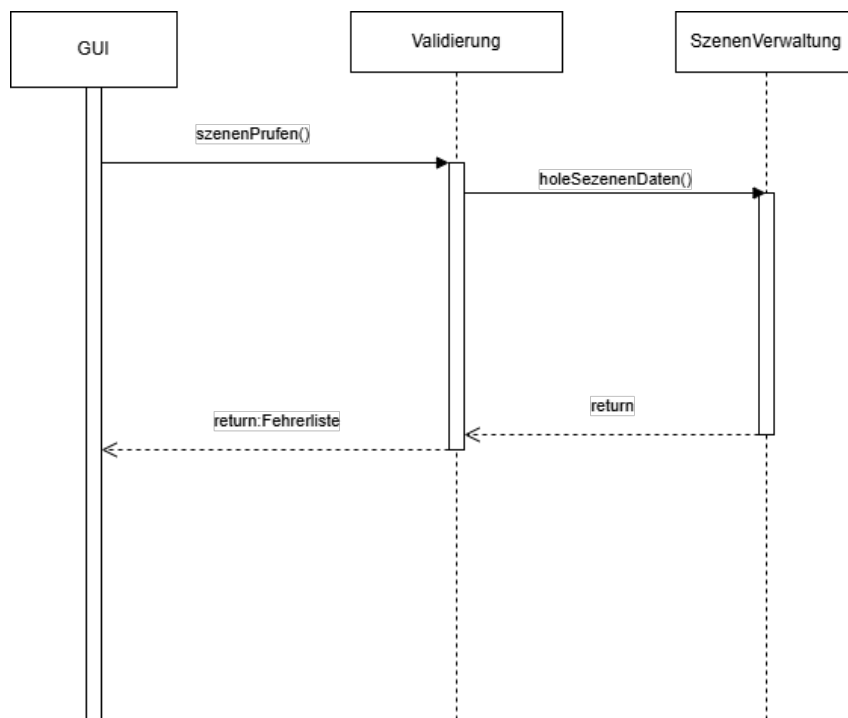


Abbildung 2.8: Ein beispielhaftes Sequenzdiagramm

Quelle der Abbildung 2.8: <http://www.uml-diagrams.org/>

3 Datenmodell

Die Anwendung SkyOrchestrator speichert die zentralen Informationen über geplante Shows, konfigurierte Drohnen, deren individuelle Flugbahnen, Formationen sowie durchgeführte Simulationen dauerhaft in einer dokumentenorientierten Datenbank. Die langfristig zu speichernden Daten werden in die Entitäten Show (E10), Drohne (E20), Flugbahn (E30), Formation (E40) und Simulation (E50) aufgeteilt.

Ein Show-Objekt enthält Metadaten zur Drohnenshow und ist über eindeutige IDs mit den zugehörigen Drohnen, Formationen und Simulationen verknüpft. Jede Drohne besitzt eine eigene Flugbahn, die wiederum eine Liste von Wegpunkten beschreibt.

3.1 Diagramm

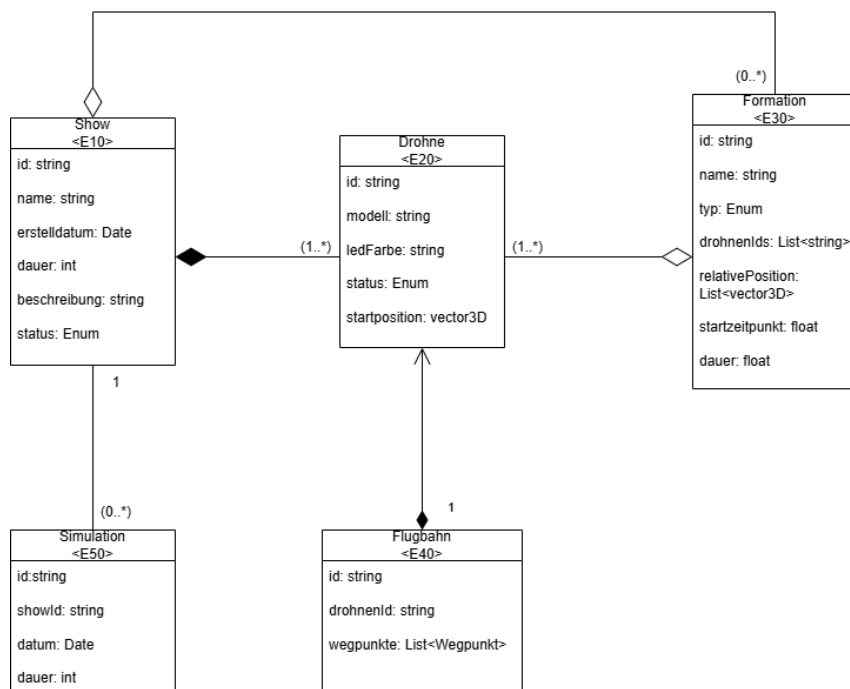


Abbildung 3.1: Ein beispielhaftes Klassendiagramm

Quelle der Abbildung 3.1: <https://app.diagrams.net//>

3.2 Erläuterung der Zusammenhänge der Klassen

In den folgenden Tabellen wird die Beziehung zwischen den Entitäten Show ⟨E10⟩, Drohne ⟨E20⟩, Flugbahn ⟨E30⟩, Formation ⟨E40⟩ und Simulation ⟨E50⟩ ausführlich dargestellt. Die Tabellen geben Aufschluss über die Art der Beziehung, die jeweilige Kardinalität, die erwartete Datenmenge sowie eine kurze inhaltliche Beschreibung. Grundlage der Modellierung sind die funktionalen Anforderungen an das System SkyOrchestrator.

Show ⟨E10⟩

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Drohne	1..*	ca. 1–300 Drohnen je Show, pro Drohne ca. 1–2 KB	Einer Show können beliebig viele Drohnen zugeordnet werden. Jede Drohne gehört eindeutig zu einer Show.
Simulation	0..*	ca. 1–10 Simulationen, pro Simulation ca. 500–800 Byte	Eine Show kann mehrfach simuliert werden. Simulationen sind eindeutig einer Show zugeordnet.
Formation	0..*	ca. 0–100 Formationen, pro Formation ca. 500–1000 Byte	Optional können Shows mehrere Formationen enthalten. Eine Formation beschreibt die gleichzeitige Anordnung mehrerer Drohnen.

Drohne $\langle E20 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Show	1	referenziert über ID	Jede Drohne ist genau einer Show zugeordnet.
Flugbahn	1	ca. 2–10 KB je Flugbahn	Jede Drohne besitzt genau eine Flugbahn, die als Liste von Wegpunkten gespeichert wird.
Formation	0..*	ca. 100–300 Byte Referenzdaten	Drohnen können in mehreren Formationen gleichzeitig oder nacheinander verwendet werden.

Formation $\langle E30 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Drohne	1..*	ca. 500–1000 Byte je Formation	Eine Formation umfasst mehrere Drohnen, deren Positionen relativ zur Formation angegeben werden.
Show	1	referenziert über ID	Jede Formation gehört eindeutig zu einer Show.

Flugbahn $\langle E40 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Drohne	1	ca. 100–1000 Wegpunkte, gesamt ca. 2–10 KB	Die Flugbahn besteht aus einer Liste zeitlich geordneter Wegpunkte mit Informationen zu Position, Zeit, Bewegung und Lichtsteuerung.

Simulation $\langle E50 \rangle$

Beziehung	Kardinalität	Erwartete Datenmenge	Beschreibung
Show	1	ca. 500–800 Byte je Simulation	Eine Simulation ist eindeutig einer Show zugeordnet. Sie enthält u.a. Zeitstempel, Dauer, Kollisionserkennung und Energieverbrauch.

4 Konfiguration

In diesem Kapitel werden die technischen Voraussetzungen und Konfigurationsschritte beschrieben, die für die Installation, Ausführung und Nutzung von SkyOrchestrator erforderlich sind. Dazu zählen unter anderem die Systemvoraussetzungen, externe Abhängigkeiten sowie Hinweise zur Installation und zum Start der Anwendung.

4.1 Technische Voraussetzungen

Für die Ausführung von SkyOrchestrator wird eine lokale Python-Installation (Version 3.13 oder höher) sowie eine funktionierende Installation der Physik-Engine PyBullet benötigt, die für die dreidimensionale Darstellung und physikalische Simulation der Drohnenszenen verwendet wird. Darüber hinaus müssen weitere externe Python-Bibliotheken installiert werden, die in der mitgelieferten Datei `requirements.txt` aufgeführt sind. Diese befindet sich im Hauptverzeichnis von SkyOrchestrator und enthält folgende Abhängigkeiten:

- `numpy`
- `pybullet`
- `pandas`
- `scipy`

Diese Bibliotheken können über den folgenden Befehl im Terminal automatisch installiert werden:

```
python -m pip install -r requirements.txt
```

Hinweis: Im Verlauf der weiteren Entwicklung kann es dazu kommen, dass zusätzliche Bibliotheken benötigt werden. In diesem Fall wird die Datei `requirements.txt` entsprechend erweitert. Es empfiehlt sich daher, vor der Installation stets die aktuelle Version dieser Datei zu verwenden.

4.2 Projektstruktur

SkyOrchestrator ist modular aufgebaut und in verschiedene Verzeichnisse unterteilt, um die Wartbarkeit und Erweiterbarkeit der Software zu erleichtern. Im Folgenden wird ein Überblick über die wichtigsten Verzeichnisse und deren Funktionen gegeben:

- `main.py` – Startpunkt der Anwendung. Über diese Datei wird die grafische Benutzeroberfläche gestartet.
- `gui.py` – Quellcode der grafischen Benutzeroberfläche (GUI).
- `sim/` – Modul zur Durchführung der Simulationen mit PyBullet. Enthält die Funktionen zur Steuerung der Drohnenbewegungen.
- `models/` – Enthält die URDF Dateien der Dronen Modelle
- `savedProjects/` – Standardverzeichnis zur Speicherung von Benutzerszenen im `.json`-Format. Diese Dateien enthalten alle Konfigurationsdaten und können jederzeit erneut geladen werden.
- `requirements.txt` – Liste der externen Abhängigkeiten, die zur Ausführung des Programms erforderlich sind.
- `README.md` – Dokumentation mit Hinweisen zur Installation, Nutzung und zu häufig gestellten Fragen.

Diese Struktur stellt sicher, dass Benutzeroberfläche, Simulationslogik und Konfigurationsdaten klar voneinander getrennt sind. Dadurch wird nicht nur die Bedienung vereinfacht, sondern auch eine spätere Erweiterung des Systems erleichtert.

4.3 Start und Nutzung der Anwendung

SkyOrchestrator wird über die Datei `main.py` gestartet. Nach dem Wechsel in das Programmverzeichnis erfolgt der Start durch Eingabe des folgenden Befehls im Terminal:

```
python main.py
```

Nach dem Start öffnet sich das Hauptmenü der grafischen Benutzeroberfläche (GUI).

Der typische Arbeitsablauf im SkyOrchestrator beginnt mit dem Anlegen eines Projekts und einer oder mehrerer Szenen. Nach Eingabe der grundlegenden Metadaten (z.B. Name, Dauer,

Drohnenanzahl) werden Formationen, Bewegungen und Effekte konfiguriert. Die fertige Szene kann gespeichert, geladen, weiterbearbeitet und schließlich simuliert werden. Die einzelnen Schritte und Abläufe sind durch Zustands- und Sequenzdiagramme in den Kapiteln 1 und 2 detailliert beschrieben.

Die Simulation wird mit Hilfe von `PyBullet` dargestellt.

4.4 Konfigurationsdateien

SkyOrchestrator verwendet zur Speicherung und Wiederverwendung von Projekten sogenannte Sitzungsdateien im `.json`-Format. Diese Dateien werden standardmäßig im Unterordner `savedProjects` des SkyOrchestrator-Verzeichnisses abgelegt und können beim Speichern frei benannt werden. Der Zweck dieser Dateien besteht darin, sämtliche Einstellungen und Parameter einer Drohnenshow dauerhaft zu sichern und zu einem späteren Zeitpunkt wieder laden zu können.

Die in der `.json`-Dateien gespeicherten Parameter spiegeln die in Kapitel 2 beschriebenen Konfigurationsschritte wider. Dazu zählen insbesondere:

- **Metadaten der Szene:** Name, Gesamtdauer, Erstellungsdatum, Autor
- **Drohnenanzahl:** Anzahl der für die Szene vorgesehenen Drohnen
- **Formationsdaten:** Ausgewählte Formationen und deren zeitliche Abfolge
- **Bewegungsabläufe:** Zuordnung von Bewegungen zu einzelnen Drohnen oder Gruppen
- **Licht- und Zeiteffekte:** Art, Farbe, Zeitpunkt, Dauer und betroffene Drohnen
- **Timeline-Events:** Steuerung von Start, Ende, Formationswechseln und weiteren Effekten

Alle Parameter werden ausschließlich über die grafische Benutzeroberfläche eingestellt (siehe Abschnitt 4.4) und in der Datei gespeichert. Eine manuelle Bearbeitung der Konfigurationsdateien außerhalb der Anwendung ist nicht vorgesehen und wird nicht unterstützt.

Weitere Konfigurationsdateien, etwa zur Steuerung der Anwendung oder zur Systemkonfiguration, sind nicht erforderlich.

4.5 Konfiguration über die Benutzeroberfläche

SkyOrchestrator ermöglicht es dem Nutzer, sämtliche Einstellungen einer Drohnenshow komfortabel über die grafische Benutzeroberfläche vorzunehmen. Alle relevanten Steuerparameter – wie Drohnenanzahl, Formationen, Lichtanimationen, Zeitabläufe und weitere Szenenparameter – können individuell angepasst werden. Die zentrale Timeline-Komponente erlaubt es, Start- und Endzeitpunkte, Formationswechsel, Lichteffekte und Landungen präzise zu planen und zu steuern.

Während der laufenden Simulation bietet die Anwendung zusätzliche Funktionen, die die Interaktion und Kontrolle über den Ablauf erleichtern. Dazu zählen die freie Kamerasteuerung, mit der die Szene aus unterschiedlichen Perspektiven betrachtet werden kann, sowie die Möglichkeit, die Show über die Timeline zu verschiedenen Zeitpunkten zu analysieren. Außerdem kann die Simulation jederzeit vorzeitig abgebrochen werden, um Anpassungen vorzunehmen oder einen neuen Simulationsdurchlauf zu starten.

Alle grundlegenden Planungs- und Simulationsfunktionen sind so gestaltet, dass sie ohne Programmierkenntnisse bedient werden können. Eine Szene kann zu jedem Zeitpunkt gespeichert werden, unabhängig vom aktuellen Konfigurationsstand. Alle Eingaben werden vor der Simulation auf Gültigkeit geprüft. Die Anwendung prüft jedoch vor dem Start der Simulation, ob alle Pflichtfelder ausgefüllt und alle Eingaben gültig sind (siehe Zustandsdiagramm zur Szenenkonfiguration, Abbildung 1.3). Fehlerhafte oder unvollständige Szenen werden durch verständliche Hinweise kenntlich gemacht und können erst nach Korrektur simuliert werden.

4.6 Mögliche Erweiterungen

Zukünftige Versionen von SkyOrchestrator könnten um eine zentrale Konfigurationsdatei ergänzt werden, in der globale Parameter wie Drohnenanzahl, Startpositionen oder Lichtfarben dauerhaft gespeichert werden. Dies würde die Wiederverwendbarkeit und Portabilität erhöhen sowie die Trennung von grafischer Benutzeroberfläche und Konfigurationslogik verbessern.

Ebenfalls denkbar ist die Erweiterung um benutzerdefinierte Presets, die als Vorlagen für neue Shows dienen, sowie eine Versionsverwaltung für Projekte. Auch die Integration von Profileinstellungen, um verschiedene Nutzerpräferenzen zu unterstützen, wäre eine sinnvolle Erweiterung.

Diese Funktionen sind zum aktuellen Zeitpunkt noch nicht umgesetzt, werden jedoch als sinnvolle Ergänzungen für eine zukünftige Weiterentwicklung in Erwägung gezogen.

5 Glossar

Effektbibliothek Sammlung vordefinierter Licht- und Spezialeffekte, die auf Drohnen angewendet werden können. Wird bei der Szenengestaltung verwendet.

Flugbahn Die zeitlich geordnete Abfolge von Positionen, die eine Drohne während einer Szene durchläuft. Flugbahnen definieren die Bewegung und Orientierung der Drohnen im Raum.

Formation Eine bestimmte räumliche Anordnung mehrerer Drohnen, die synchronisiert ausgeführt wird. Formationen werden vom Benutzer ausgewählt oder definiert und bestimmen visuelle Muster in der Show.

GUI (Graphical User Interface) Die grafische Benutzerschnittstelle, über die Nutzer mit dem SkyOrchestrator interagieren. Sie dient zur Eingabe von Szenenparametern, zur Steuerung der Showkonfiguration und zur Anzeige von Rückmeldungen.

JSON (JavaScript Object Notation) – textbasiertes Datenformat zur strukturierten Speicherung und zum Datenaustausch, auch für Projektexporte verwendet.

URDF (Unified Robot Description Format) – XML-basiertes Format zur Beschreibung von Roboter-Modellen.

Projekt Eine vollständige Zusammenstellung von Szenen, Formationen und Konfigurationen, die gemeinsam gespeichert und wiederverwendet werden kann.

ProjektManager Modul zur Organisation und Speicherung kompletter Projekte. Er koordiniert das Laden, Speichern und Exportieren von Projektdaten.

ShowKonfigurator Eine zentrale Softwarekomponente zur Verwaltung und Bearbeitung von Szenen innerhalb eines Projekts. Er vermittelt zwischen GUI, Szenenverwaltung und weiteren Modulen wie der Formations- oder Effektbibliothek.

Simulation Die visuelle und logische Darstellung einer konfigurierten Szene mit Hilfe eines 3D-Simulators (z. B. PyBullet), um die geplante Show zu testen und zu validieren.

Szene Die kleinste logische Einheit einer Show, bestehend aus Drohnenformationen, Bewegungsabläufen, Licht- und Zeiteffekten. Szenen können unabhängig bearbeitet und simuliert werden.

Validierung Überprüfung der Szenen auf Vollständigkeit und Konsistenz. Das Modul erkennt fehlerhafte Eingaben oder Kollisionen und gibt strukturierte Rückmeldungen an die GUI.